A Layered Architecture for Digital Twins with Progressive Fidelity: A Case Study in Airport Operations with Specification-First Development

Arthur Chevallier and German Castignani

Luxembourg Institute of Science and Technology
arthur.chevallier@list.lu, german.castignani@list.lu

Abstract. Digital twins of complex cyber-physical systems rarely start at full resolution: they grow from simple representations towards higher spatial, temporal, and behavioural fidelity by integrating physical layouts, real-world data, and decision support. Existing architectures decompose a twin into layers, but do not explicitly govern how fidelity can increase without invalidating existing capabilities. We propose a four-layer architecture in which each layer exposes an explicit contract, so raising fidelity is governed as an extension rather than an implicit rewrite. The topmost layer is decisional, with actions returning to the source system through approval gates. We use a specification-first methodology in which event schemas, data models, and service specifications define the intended boundaries. The architecture is instantiated in VirtualSky International Airport (EVSK), an open-source, ongoing research prototype comprising nine domain services and one API gateway. A repository-history analysis examines the blast radius of four fidelity upgrades, while a conformance audit identifies where specifications alone were insufficient. The case evidence supports append-only inter-layer contracts, backed by contract and regression tests, as a practical principle, while also exposing one lower-layer dependency that remains to be refactored.

Keywords: Digital twins · Progressive fidelity · Layered architecture · Airport operations · Specification-driven development · Decision support.

1 Introduction

Digital twins are virtual counterparts of physical systems maintained through structured data flows [1,2]. In complex domains such as aviation, they cannot be built at maximum resolution from the outset: data, models, and integrations become available incrementally. A project may begin with a logical event model and later add physical simulation, geospatial representation, live data, analytics, and decision support. Throughout this paper, *fidelity* denotes how closely a twin matches its physical counterpart along spatial, temporal, and behavioural dimensions (Section 3), thereby supporting more reliable feedback and decision making. Increasing any dimension can alter representations, interfaces, and data

flows already used by earlier capabilities. Extension can therefore become restructuring, hindering long-term evolution and reproducibility [3].

Existing frameworks provide important decompositions. ISO 23247 defines a manufacturing reference architecture [4], Tao et al. separate the physical entity, virtual entity, data, services, and connections [5], and Redelinghuys et al. separate concerns from acquisition to emulation [6]. They do not explicitly make *fidelity-preserving evolution* a constraint: higher-fidelity capabilities should extend the contracts of lower layers rather than restructure them. This is the gap investigated here.

We therefore ask:

> **RQ1: How can a digital-twin architecture support progressive increases in fidelity while preserving the contracts and functionality of previously implemented layers?**

We study EVSK, the VirtualSky International Airport digital twin and an ongoing research prototype. Its operational entities and ground truth are synthetic, except for opt-in L2 live ADS-B and METAR feeds. Airports provide a demanding testbed because aircraft, vehicles, passengers, baggage, infrastructure, schedules, weather, and disruptions interact across multiple operational timescales. Existing research provides mature models for individual aspects of this system, including delay propagation [7], delay cost [8], passenger-flow prediction [9,10], and airport digital twins [11,12]. EVSK instead uses the airport as an instrument for studying whether progressively richer representations can share stable contracts. Its implementation comprises nine FastAPI domain services behind one API gateway, including a decisional layer for anomaly detection, scenario analysis, optimisation, appraisal, and gated recommendations.

The second question concerns how such an architecture can be implemented and maintained without its contracts eroding. Event schemas, shared concepts, service boundaries, and data flows must remain coherent as a distributed system evolves, and the local correctness of individual changes does not guarantee this. We therefore use a specification-first methodology in which service specifications, event schemas, data models, and simulation rules are defined before implementation and remain authoritative, supplemented by reusable "skill files" encoding implementation patterns and known pitfalls. Implementation was assisted by coding agents; we treat this as operational context, and Section 4 delimits where human intervention was required. This motivates:

> **RQ2: To what extent does specification-first development preserve architectural contracts during sustained implementation of a progressively evolving digital twin, and where does it require independent verification?**

The paper makes four contributions:

1. a layered fidelity architecture that treats progressive, contract-preserving evolution as an explicit architectural principle, with fidelity defined along spatial, temporal, and behavioural dimensions;

2. a specification-first development methodology for maintaining these architectural contracts during implementation;
3. EVSK, a fully open-source airport digital-twin prototype that instantiates the architecture across four layers, including established analytical and ML components used to exercise the layer boundaries; and
4. an empirical examination of the principle, combining an analysis of how changes propagated across layers with an audit of where implementation departed from specification.

The research object is therefore not the act of implementing an airport application, nor a new machine-learning method. It is the architectural principle and its empirical examination; EVSK is the single-case instrument through which contract preservation and violations are observed. The methodology is the engineering discipline used to maintain the principle, together with evidence of where specifications alone were not sufficient.

The remainder of the paper is organised as follows. Section 2 discusses related work. Section 3 presents the layered fidelity architecture, Section 4 describes the development methodology, and Section 5 presents EVSK. Section 6 reports the evaluation, followed by discussion and conclusions in Section 7.

2 Related Work

Digital-twin research provides several foundations. Kritzinger et al. [13] distinguish models, shadows, and twins by automated exchange. ISO 23247 specifies manufacturing-domain entities, functional entities, and exchanges [4]; Tao's five-dimensional model organises physical entity, virtual entity, data, services, and connections [5]; and three-layer IoT architectures separate perception, network, and application [14]. These frameworks permit modular implementation, but do not state backward-compatible fidelity evolution as an invariant. For example, replacing EVSK's weather event with provider-specific METAR fields would force capacity and analysis consumers to change; the cited frameworks locate the affected components but do not forbid that coupling. EVSK instead preserves a canonical event behind an adapter. We compare constraint coverage, not claim that earlier architectures must fail; quantifying them would require comparable longitudinal implementations.

Airport modelling provides the application context for examining this question. Analytical and simulation-based research has addressed delay propagation [7] and the cost of delay [8], while machine learning has been applied to passenger-flow and boarding-time prediction [9,10]. More recent work explicitly applies digital-twin concepts to airport operations, including turnaround operations [11], landside simulation [12], and airport-wide adoption guidance [15]. These studies demonstrate the feasibility and value of airport digital twins, but generally address particular operational scopes or fixed capabilities rather than the problem of evolving a twin across fidelity levels without restructuring previously implemented functionality. EVSK uses the domain as a demanding in-

stantiation of the architectural principle rather than as the primary research contribution.

AI-assisted software-engineering studies evaluate repository-level issue resolution [16]. We do not evaluate agent performance; RQ2 instead asks whether a specification-first process preserves the contracts introduced by RQ1, and where independent verification remains necessary.

3 Layered Fidelity Architecture

3.1 Fidelity and why it must be progressive

We define the *fidelity* of a digital twin along three dimensions: *spatial granularity*, from topological relations between entities to gate-level coordinates and geometry; *temporal resolution*, from discrete simulated events to sub-second live updates; and *behavioural realism*, from discrete state transitions to physically grounded and data-calibrated dynamics. In complex domains such as aviation, a twin cannot be built at maximum resolution from the start. The required data, models, and integrations become available over time, and each must be added to a system that already works. Fidelity is therefore acquired progressively, and the architectural question is whether each increase can be made without restructuring what is already implemented.

3.2 Four capability layers

The architecture is a stack of four capability layers, each of which raises fidelity along a different combination of the three dimensions (Table 1):

- **L0, logical event-driven simulation.** Domain entities are represented as state machines that communicate purely through events. There is no geometry and no external data at this layer. Something happens, an event records it, and downstream consumers react.
- **L1, physical layer.** Spatial layout is added on top of L0: positions, distances, and the travel or transfer times they induce between entities already defined at L0.
- **L2, geospatial and real-world data layer.** The twin is connected to the outside world. Map-based visualisation, live external feeds, and reference data ground the logical and physical state in observed reality.
- **L3, decisional layer.** Scenario analysis, anomaly detection, optimisation, appraisal, and recommendation are built on the first three layers. Actions reach the source system only through approval gates.

Table 1. Fidelity contribution of each layer along the three dimensions.

Layer	Spatial	Temporal	Behavioural
L0	topological	discrete events	state transitions
L1	gate-level geometry	unchanged (discrete)	travel and transfer times
L2	georeferenced	live and historical feeds	data-calibrated
L3	asset-level decisions	scenario horizons	prescriptive, gated actions

At L3, the spatial gain is not a richer rendering but the ability to target decisions to represented assets, such as a terminal, gate, runway, or flight. Three-dimensional visualisation could increase representational spatial fidelity, but would belong to L1 or L2 unless it formed part of a decision interface; because EVSK does not currently implement it, Table 1 does not use it as evidence.

Figure 1 summarises this domain-independent stack and shows how its four layers map to the EVSK airport implementation.

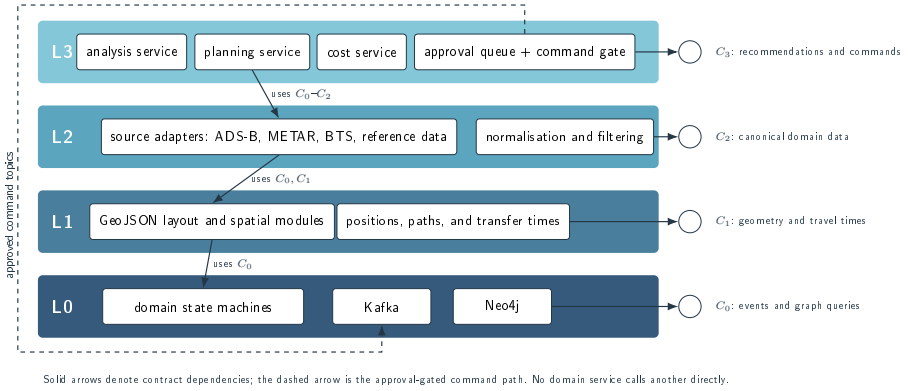


Fig. 1. UML-style component-and-provided-interface view of EVSK's four layers. Solid arrows are dependencies on contracts C_i; only the dashed, approval-gated command path closes the loop. Fidelity upgrades may extend a contract but may not alter an existing element.

L0 to L2 form a digital shadow: data flows from the physical system, or from its simulated stand-in, into the twin [13]. L3 can close the loop through the command gate. Figure 1 uses standard component and provided-interface idioms to make allowed dependencies reviewable by humans and tools; it is an architectural view, not an executable UML model, and we do not claim that the notation itself improved coding-assistant performance.

A separate axis governs how a single layer matures, independent of which layer it is. Capability within a layer deepens along a fixed ladder, from visualisation, to monitoring and alerting, to prediction, to prescription, to actionability.

This ladder describes what a layer *does*, whereas fidelity describes how closely it *represents*; a layer counts as mature once it has climbed the ladder, not merely because it renders something.

3.3 Contracts and the fidelity-preservation invariant

Each layer L_i exposes a contract C_i to the layers above it: the event schemas it publishes, the graph model it owns, and the service interfaces it offers. Two rules govern dependencies. A layer may depend on the contracts of lower layers but not on their internals, and lower-layer modules must not import higher-layer internals. Higher layers may influence operational behaviour only through declared lower-layer input or command ports. In particular, L3 may read from L0–L2, while approved actions return through command topics rather than direct imports, keeping the decisional layer removable from the shadow underneath.

These rules yield the architectural invariant on which the rest of the paper rests:

> **Fidelity-preservation invariant.** Raising the fidelity of layer L_i may extend its contract C_i, but must not modify or remove any existing element of any contract C_j. Consequently, layers above L_i continue to operate unchanged.

For example, replacing synthetic weather with a live METAR feed changes the implementation of an L2 adapter, but the events that L3 consumes keep the same schema. Upgrading a layer's fidelity thus changes that layer's modules and, at most, adds to its contract, and it does not trigger refactoring in the layers that consume it. Section 6 tests this property against the development history of EVSK.

3.4 The L2 to L3 boundary and current approval gate

Because L3 can trigger actions, corrupted data crossing from L2 is consequential. EVSK currently applies source-specific parsing and filtering in its adapters (including required-position and radius checks for ADS-B), canonical envelope construction for Kafka events, Pydantic validation at REST boundaries, and event-ID deduplication in consumers. These checks reduce malformed and duplicate input, but they are not a single end-to-end integrity gate: there is no runtime schema-registry validation, uniform plausibility policy, or source provenance score across all feeds. Moreover, the prototype planning-model trainer reads the BTS extract directly rather than through the planning adapter. We therefore exclude those prototype predictors and their scores from the evaluation and treat adapter-only ingestion plus runtime schema validation as future hardening.

The approval gate is a bounded prototype, not a complete safety case. It blocks configured actions, skips low-confidence ones, auto-approves eligible actions, and holds safety-guarded actions for an approver role. Only concrete

supported mappings—currently flight holds and gate reassignments—produce domain commands; aggregate recommendations remain audit records and the queue is in memory. Durable proposals, broader mappings, calibrated confidence and provenance, and stronger policy separation remain future work.

3.5 Generality

What the architecture fixes is the order in which fidelity is added and the contracts between layers, not the domain those layers describe. A hospital campus twin could follow the same order: patient, staff, and equipment state machines on an event bus at the base, ward and building layout above, real-time location and environmental sensing next, and bed-allocation or staffing optimisation at the top. The same layering applies to smart ports, manufacturing plants, and urban mobility networks. We use an airport, EVSK, as the concrete instantiation and stress test in the remainder of the paper.

Section 5 shows how EVSK realises each layer with a concrete technology stack, together with the architectural invariants that keep them composable in practice.

4 Enforcing Layer Contracts with Specification-First Development

The layered architecture in Section 3 keeps fidelity additions from forcing a rewrite by fixing what each layer may depend on. That guarantee holds only if the implementation actually respects those contracts, and nothing about the architecture enforces that on its own. The methodology that keeps the two aligned rests on an inversion of the usual workflow: the specification is written first, is machine-readable, and is authoritative. Implementation may not deviate from it, and where reality demands a change, the spec is updated first and the code follows. Before any service is implemented, four definitions already exist: the event-bus schemas, the graph data model, the simulation rules, and one specification per service describing its endpoints, events, state machines, and ownership boundaries. Each new layer, in practice, starts as a specification of the contract it will expose to the layer above and consume from the layer below, before a single line of that layer's code is written.

This discipline has an intellectual lineage older than coding assistants. Writing the authoritative description before the implementation echoes formal specification traditions [17], and decomposing the system so that each service hides exactly one set of design decisions follows Parnas' information-hiding criteria [18]. Because the specifications are machine-readable, they can also serve as the context for automated coding assistance, which made drift between specification and code more costly than in a purely manual workflow.

Complementing the per-service and per-layer specifications are "skill files", which encode reusable project knowledge in a form readable by both humans

and coding assistants. Each skill file has three parts: concept explanation, copy-paste patterns, and gotchas. The patterns keep new code consistent with the project's idioms, and the gotchas encode failures that review caught the hard way, so the same mistake is not repeated. Cross-cutting skills exist for exactly the four runtime invariants of EVSK (Section 5): the simulation clock, the event bus, the graph as single source of truth, and the no-silent-mutation rule. These are the invariants most likely to be violated under time pressure, since violating any one of them is often the fastest way to make a feature appear to work. Each service additionally owns one skill file documenting its own state machines and quirks.

Entry-point files (`CLAUDE.md`, `AGENTS.md`, and editor rule files) instruct the assistant to load the relevant specification and skill file before writing code. Hard rules act as guardrails, and each one maps directly onto a layer contract rather than a general best practice: no wall-clock time in business logic, no service-to-service HTTP calls, no silent mutations, and all operational entities and ground truth synthetic except opt-in L2 live feeds. Because each rule corresponds to a specific invariant, a violation is detectable by search or contract test, rather than surfacing later as a layer-boundary failure that looks unrelated to its cause.

Role of coding assistants and the human perimeter. EVSK was implemented with the help of AI coding assistants operating inside this specification framework. We treat this as operational context and not as a contribution, and we delimit it precisely. The assistants produced code within the boundaries set by the specifications. Humans were responsible for everything that defines those boundaries: writing and approving the specifications and event contracts, deciding the hard rules, and reviewing every change against the layer contracts. Human intervention was also required to correct assistant output that satisfied the immediate task but not the architecture. A mid-project audit (Section 7) found three such cases: duplicated constants that had diverged, recommendations that were never connected to command topics, and an ML endpoint that reported success without having trained a model. In each case the code conformed to a local reading of the task but not to the contract, and the fix was made by a human reviewer and, where relevant, promoted into a skill file as a gotcha.

Periodic post-mortems close the loop: defects and near-misses found in review, including layer-boundary violations that slipped past the hard rules, are promoted back into skill files, so the knowledge base accumulates detail where an added layer most recently threatened the contract beneath it. Progressive fidelity therefore has to be achieved twice: once in what the twin models, and again in how faithfully the implementation matches what was specified.

5 EVSK: Instantiating the Layered Architecture

EVSK instantiates the architecture as nine FastAPI domain services behind a thin Node.js/Express API gateway, with React/TypeScript dashboards, Kafka as the event backbone [19], Neo4j as the graph store [20], an observability stack,

and staged CI. The conventional stack keeps the evaluation focused on the architectural claim rather than on a novel platform.

Each layer is realised with a concrete slice of this stack. L0 uses Kafka and Neo4j for the domain state machines. L1 adds terminal, gate, runway, and taxiway geometry. L2 adds map visualisation, an ADS-B overlay, live and historical METAR weather, BTS schedule calibration, and OurAirports/OpenFlights reference data. L3 adds what-if and Monte-Carlo scenario analysis, anomaly detection, recommendations, and discounted-cash-flow investment appraisal on Eurocontrol Standard Inputs [21]. The planning service also contains demand and delay predictors, but their present training pipeline is a prototype: the demand feature set leaks its target and the delay labels are rule-derived. We report neither their error nor discrimination as evidence of predictive skill.

Four runtime invariants are intended to keep these layers composable. Services use the simulation clock rather than wall-clock time for business logic; a single orchestrator broadcasts ticks on a dedicated topic (see Annex, Fig. 7). EVSK can run at configurable speed up to 3600× real time and supports deterministic replay when fixtures, seeds, and event order are held fixed. Operational event topics carry facts, while commands use separate topics. The graph store is the source of truth for structural state, each service owns its node labels, and in-memory caches are intended to be rebuildable after restart. The no-silent-mutation rule requires modelled state changes to emit events. These are conformance targets checked by review and tests, not guarantees supplied by the technology stack.

EVSK's build history provides case evidence for this composability: the system moved from pure simulation, through map and live-data integration, to scenario analysis and financial planning without re-platforming the stack. Section 6 audits the stronger contract-isolation claim and reports where it did not fully hold.

Domain depth is where the implementation earns the word "twin". The flight domain runs a nine-state lifecycle state machine with runway queueing and delay cascades that propagate up to five hops, mirroring network-level propagation models [7] (see Annex, Fig. 5). Passenger flow contains a LightGBM model that predicts security-queue depth ninety minutes ahead [9,10]. We classify this predictor as an embedded L0 component: on each simulation tick its output enters the security-throughput calculation, changing how many passengers transition from the security queue to airside. The write-back is therefore an internal, event-emitting L0 feedback loop, not a dependency from L0 on L3. Baggage travels a thirty-zone conveyor network with dangerous-goods screening, and weather is a four-state finite-state machine whose runway-capacity impacts cascade into the flight domain (see Annex, Fig. 6). A rule-based incident engine cascades five hazard types through the graph under explicit depth limits.

The twin is open at its configuration and data boundaries. Airport identity, terminals, gates, runways, operational constants, and cascade limits are loaded from YAML rather than embedded in service logic. A shared registry supplies a common adapter protocol in the simulation orchestrator, weather service, and

passenger service; the planning service uses a separate local adapter factory. Within those three adopters, a source can be switched without changing its downstream consumer. The scoped Open/Closed claim is therefore that configuration and adapters extend the represented world while tested domain logic remains unchanged.

The simulated day is operationally plausible at scale: 420 flight movements, roughly 30,000 passengers, and roughly 36,000 baggage items, driven by the simulation clock described above.

The decisional layer is implemented as part of the current research prototype. The cost service maintains profit-and-loss statements, EC 261/2004 liability [22], and carbon accounting. The planning service runs a Monte-Carlo infrastructure scenario engine producing NPV, IRR, and payback appraisals. The analysis service provides Isolation-Forest anomaly detection, slot-allocation MILP, and recommendations routed through the approval queue. Concrete supported actions use command topics; other recommendations are presently audit-only, so command coverage remains incomplete.

To support reproducibility, specifications, skill files, infrastructure definitions, data pipelines, and the implementation are available in the project repository. Airport identity and much of the operational structure are configuration-driven; adapting EVSK to another airport is intended to require configuration and adapter changes rather than a separate codebase, although this has not yet been validated in a second deployment.

6 EVSK: Evaluation of progressive fidelity in our use-case

This section examines progressive fidelity in EVSK. Because the repository has no prospective milestone ledger, this is a retrospective path-level case analysis rather than a controlled benchmark.

6.1 Audit design

The unit of analysis is an upgrade path, not a raw commit count. We traced L1 geometry, L2 live feeds, BTS-based calibration, and L3 cost analysis from their introducing diffs to the current implementation, specifications, schemas, and tests. The reproduction script `blast_radius_metrics.py` applies declared path scopes to the introducing changesets and uses Git's diff tree to count unique files, pre-existing files modified, and top-level components touched. The counts complement the contract audit and describe change spread, not productivity.

Table 2. Observed contract and consumer effects of four fidelity upgrades.

Upgrade	Touched subsystem	Contract and consumer effect
L1 geometry	Layout fixture, spatial utilities, domain transitions	Added geometry; L0 contracts unchanged
L2 live feeds	ADS-B/weather adapters and source UI	Existing domain representations; consumers unchanged
Data calibration	BTS parser, schedule and passenger generators	Contracts unchanged; L0 internals edited
L3 cost analysis	New consumer, persistence, API, dashboard	New cost contract; existing payloads unchanged

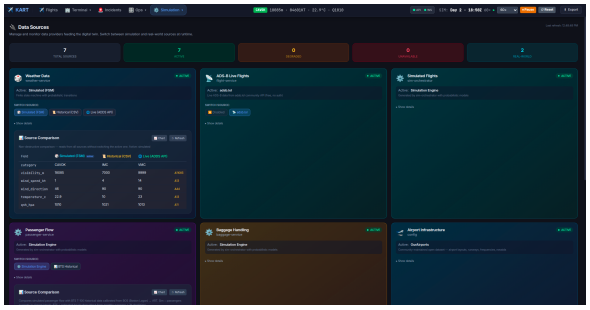


Fig. 2. L2 implementation view used in the audit: the source-selection dashboard. The shared registry is used by the simulation orchestrator, weather service, and passenger service; planning adapters use a separate local factory.

6.2 Results across upgrade paths

Figure 2 shows the L2 source-selection surface; the L0 graph and L3 cost views are retained in the Annex (Figs. 3 and 4). Table 2 reports what changed underneath them. The automated radii—unique scoped files / pre-existing files modified / components touched—are 14/6/5 for L1 geometry, 17/9/7 for L2 live feeds, 7/5/3 for BTS calibration, and 35/12/6 for L3 cost analysis.

L1 geometry and L2 live feeds added geometry, adapters, and presentation components while retaining operational event shapes. L3 cost analysis added persistence, an API, and a dashboard, but remained interface-additive: it consumed existing facts and published a new contract without changing an existing producer or consumer.

The BTS path is a counterexample to strict implementation isolation. External contracts stayed stable, but calibration added imports and branches inside the L0 schedule and passenger generators. It preserved consumer compatibility while editing lower-layer code; a cleaner design would materialise calibrated configuration through a declared L0 input port. Thus the audit supports preservation of existing contract elements, not the stronger claim that fidelity upgrades never edit lower-layer code.

6.3 Specification conformance

Event-bus and data-model specifications changed with the cost and command-topic code. Contract tests cover envelopes and handoffs; fidelity tests exercise runway and investment assumptions; approval tests cover classification, transitions, and supported commands. They do not establish source truthfulness, end-to-end availability, or autonomous-operation safety, so the BTS dependency and earlier audit findings still require code inspection.

7 Discussion and Conclusion

The results address the two research questions directly. **RQ1** asked whether fidelity can be increased while preserving previously implemented layers. All four audited paths preserved existing event and service contracts, and no existing consumer required an interface migration. Three paths were additive at both the interface and module levels; BTS calibration edited lower-layer generators and therefore exposes a conformance gap. The supported conclusion is consequently about contract preservation, not zero change to lower-layer implementation.

RQ2 asked whether specification-first development can preserve these contracts during sustained implementation, and where it requires independent verification. The experience was useful but insufficient on its own. A mid-project audit found diverging duplicated constants, recommendations disconnected from command topics, and an ML endpoint that reported success without training a model. Cross-service contract tests now check event-envelope fields and producer/consumer handoffs, while fidelity regression tests exercise planning assumptions. These executable checks and periodic audits are necessary complements to the specifications.

The findings are complementary: *the architecture defines what must remain stable; the methodology helps preserve it.* Neither is sufficient alone. Contracts, specifications, executable checks, and dependency audits must operate together. Existing reference architectures could adopt the same invariant; EVSK's distinction is to state it as a rule and audit concrete upgrade paths against it.

Ports, hospitals, and warehouses could use the same ordering of event state, spatial representation, observed data, and decisions. Transfer remains a proposition: this is one retrospective case, one small team, and synthetic ground truth [23]. We did not reimplement EVSK under ISO 23247, Tao's model, or a three-tier IoT architecture. Comparative superiority, productivity effects, and defect-rate improvements therefore cannot be inferred.

Future work will invert the BTS calibration dependency, persist approval proposals, extend command-topic coverage, add runtime schema and provenance validation, and test the architecture in a second domain or federated multi-airport setting. Overall, EVSK provides qualified support for **RQ1** at the contract level and supports **RQ2** by showing that specification-first development helps maintain boundaries but requires independent verification.

Acknowledgments and use of AI. No specific funding was received. AI tools assisted language editing; the authors remain responsible for the content.

References

1. Glaessgen, E., Stargel, D.: The digital twin paradigm for future NASA and US air force vehicles. In: 53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference (2012)
2. Grieves, M., Vickers, J.: Digital twin: mitigating unpredictable, undesirable emergent behavior in complex systems. In: Transdisciplinary Perspectives on Complex Systems, pp. 85–113. Springer (2017)
3. Jones, D.E., Snider, C., Nassehi, A., Yon, J.M., Hicks, B.J.: Characterising the digital twin: A systematic literature review. CIRP Journal of Manufacturing Science and Technology (2020)
4. International Organization for Standardization: ISO 23247-2:2021 automation systems and integration — digital twin framework for manufacturing — part 2: Reference architecture. Tech. rep., ISO (2021)
5. Tao, F., Cheng, J., Qi, Q., Zhang, M., Zhang, H., Sui, F.: Digital twin-driven product design, manufacturing and service with big data. The International Journal of Advanced Manufacturing Technology 94(9–12), 3563–3576 (2018)
6. Redelinghuys, A.J., Basson, A.H., Kruger, K.: A six-layer architecture for the digital twin: A manufacturing case study implementation. Journal of Intelligent Manufacturing (2020)
7. Pyrgiotis, N., Malone, K.M., Odoni, A.: Modelling delay propagation within an airport network. Transportation Research Part C: Emerging Technologies 27, 60–75 (2013)
8. Cook, A., Tanner, G., Williams, V., Meise, G.: Dynamic cost indexing – managing airline delay costs. Journal of Air Transport Management 15(1), 26–35 (2009)
9. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y.: LightGBM: a highly efficient gradient boosting decision tree. In: Advances in Neural Information Processing Systems. vol. 30 (2017)
10. Schultz, M., Reitmann, S.: Machine learning approach to predict aircraft boarding. Transportation Research Part C: Emerging Technologies 98, 391–408 (2019)
11. Conde, J., Muñoz-Arcentales, A., Romero, M., Rojo, J., Salvachúa, J., Huecas, G., Alonso, Á.: Applying digital twins for the management of information in turnaround event operations in commercial airports. arXiv preprint arXiv:2408.14291 (2024)
12. Palaşcă, A., Semenescu, A., Stăncel, I.N.: Using digital twin in simulating airport landside operations. Annals of the Academy of Romanian Scientists Series on Engineering Sciences 17(2), 137–148 (2025)
13. Kritzinger, W., Karner, M., Traar, G., Henjes, J., Sihn, W.: Digital twin in manufacturing: A categorical literature review and classification. IFAC-PapersOnLine 51, 1016–1022 (2018)
14. Sethi, P., Sarangi, S.R.: Internet of things: Architectures, protocols, and applications. Journal of Electrical and Computer Engineering 2017, 9324035 (2017)
15. Transportation Research Board: Guidebook for digital twin programs at airports (ACRP project 03-66) (2024)
16. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: can language models resolve real-world GitHub issues? arXiv preprint arXiv:2310.06770 (2024)
17. Lamport, L.: Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers. Addison-Wesley (2002)

18. Parnas, D.L.: On the criteria to be used in decomposing systems into modules. Communications of the ACM 15(12), 1053–1058 (1972)
19. Kreps, J., Narkhede, N., Rao, J.: Kafka: a distributed messaging system for log processing. In: Proceedings of the NetDB Workshop (2011)
20. Angles, R., Gutierrez, C.: Survey of graph database models. ACM Computing Surveys 40(1), 1–39 (2008)
21. EUROCONTROL: Standard inputs for EUROCONTROL cost-benefit analyses (2024), https://www.eurocontrol.int
22. European Parliament and Council: Regulation (EC) No 261/2004 establishing common rules on compensation and assistance to passengers in the event of denied boarding and of cancellation or long delay of flights (2004), official Journal of the European Union, L 46
23. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 14, 131–164 (2009)

Annex

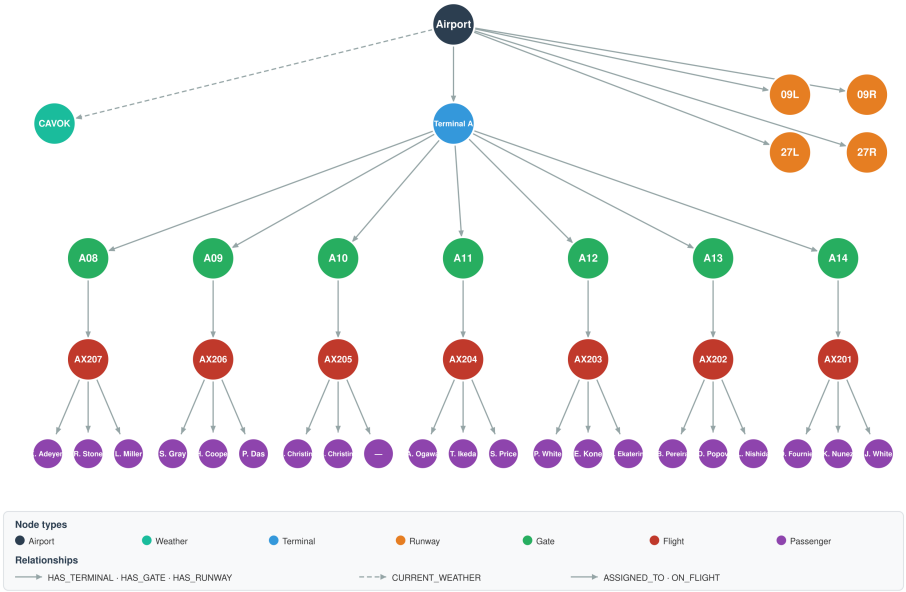


Fig. 3. L0 implementation view: Neo4j stores the operational graph reconstructed from domain events.

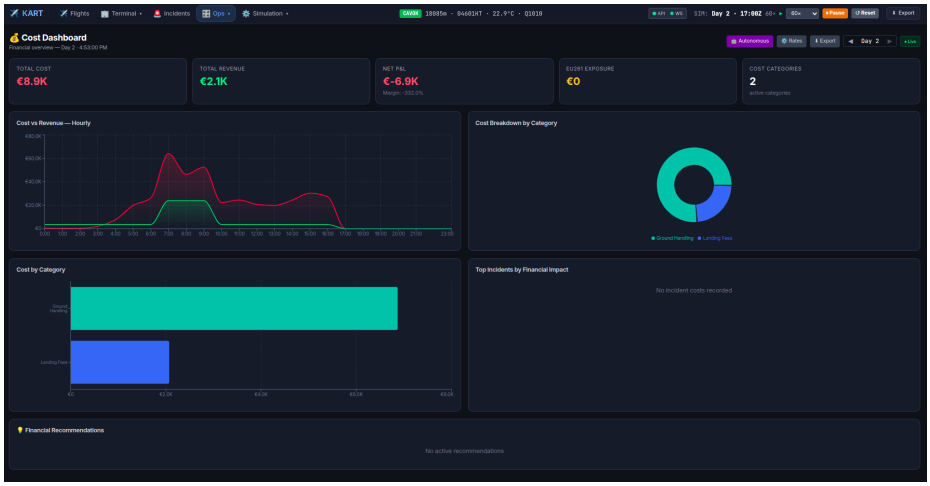


Fig. 4. L3 implementation view: the cost dashboard consumes operational events to present profit-and-loss and cost categories.

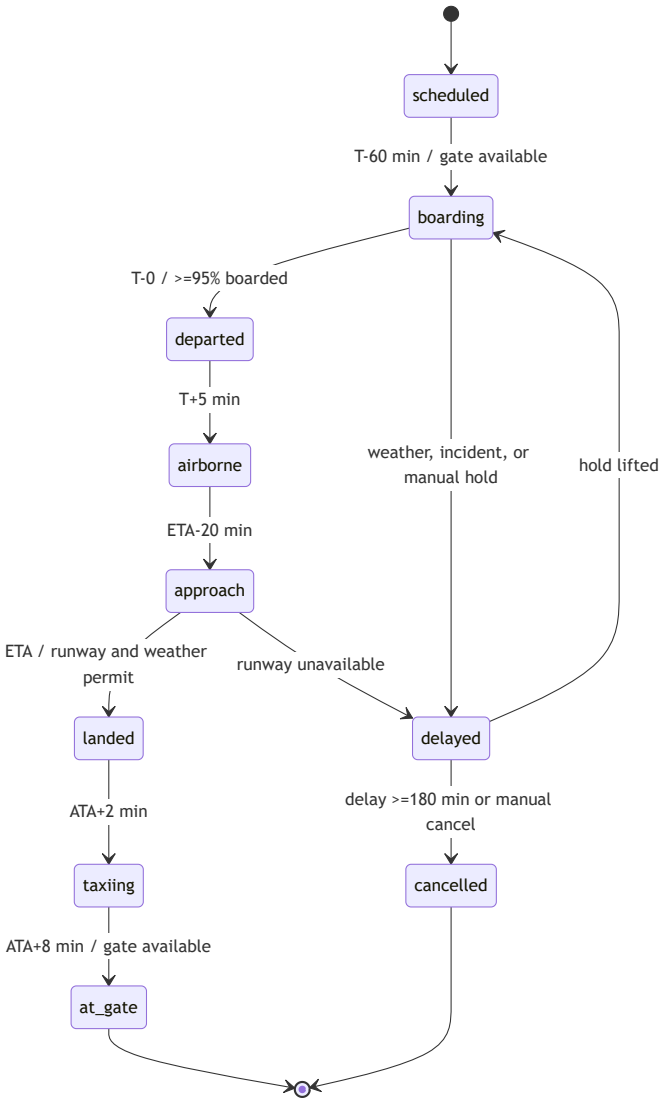


Fig. 5. Flight lifecycle driven by simulated time and constrained by gates, runways, weather, incidents, and boarding progress.

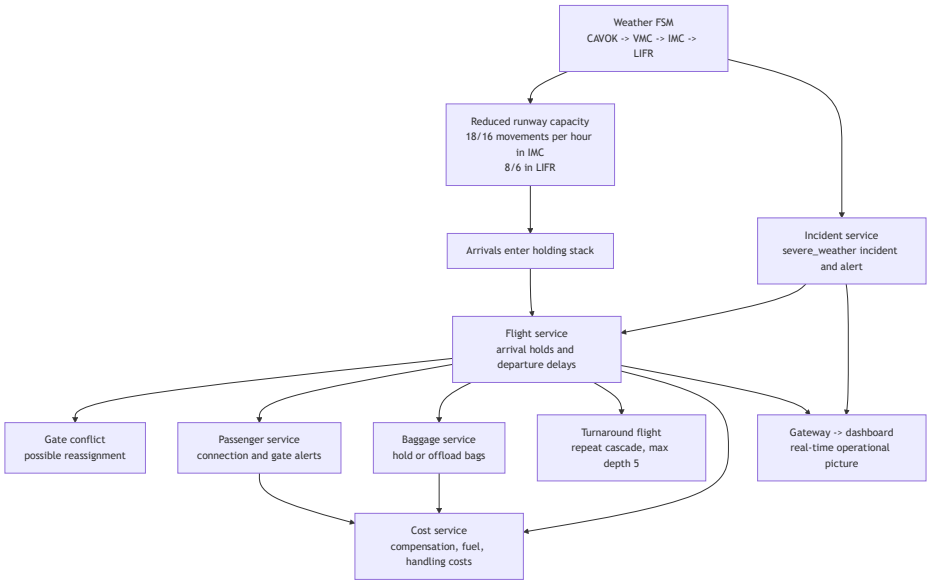


Fig. 6. Causal propagation from weather degradation to runway capacity, flight delays, passenger and baggage disruption, and operational cost.

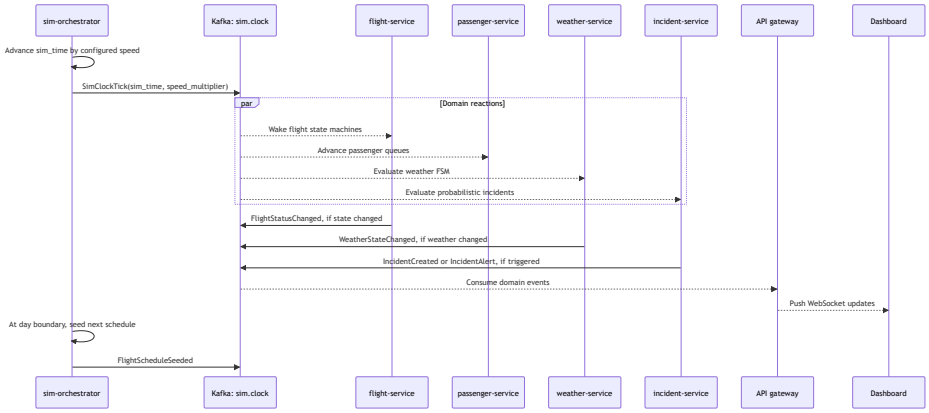


Fig. 7. Simulation-clock coordination in EVSK. A single virtual-time stream drives every service, and resulting facts are pushed to the dashboard through the gateway.